



Speech Model

Outcomes

1. Whisper Model
2. ROS 2 Recording Publisher (RecordingPublisher node)
3. ROS 2 Word Publisher (Words node)
4. ROS 2 Word Subscriber (Speak node)
5. ROS 2 Launch Files

Quiz Revision

- ROS 2 communicate with each other using topics, services, actions, and parameters to build a complete robotic system.
- A ROS 2 allows publishers and subscribers to exchange messages asynchronously in a continuous stream.
- ROS 2 are used when a node needs to send a request and wait for a single response from another node.
- in ROS 2 are designed for long-running tasks and provide feedback, status updates, and a final result.
- each ROS 2 node maintains its own set of....., which act like configurable settings that control its behavior.

Notes in Assignment 2

- **GitHub** Is a cloud platform for storing and managing your code.
Uses **Git**, the version-control system that tracks all changes in your project.

1) Clone

Run the following commands (use the actual URL):

```
cd ros2_ws/src  
git clone git@github.com:gitalg/...  
cd assignment-2-ros-2-and-vision-<yourGitHubname>
```

2) Push

you need to be in `~/ros2_ws/src/assignment-2-ros-2-and-vision-<yourname>` when you run these commands:

```
git add aisd-vision aisd-motion (select the targeted files)  
git commit -m "created empty aisd-vision and aisd-motion packages" (save in your machine)  
git push " (save in the github online)
```

Notes in Assignment 2

- After you build the packages and push it in github your repo should be as follows

Name	Last commit message
 ..	
 aisd_vision	adding comments to the codes, hand.py and move.py
 resource	creating empty aisd_vision and aisd_motion
 test	creating empty aisd_vision and aisd_motion
 LICENSE	creating empty aisd_vision and aisd_motion
 package.xml	creating empty asid_hearing and aisd_speaking packages
 setup.cfg	creating empty aisd_vision and aisd_motion
 setup.py	completing the package.xml and setup.py TODOs

Notes in Assignment 2

- After you build the packages and push it in github your repo should be as follows

Name	Last commit message
 ..	
 aisd_vision	adding comments to the codes, hand.py and move.py
 resource	creating empty aisd_vision and aisd_motion
 test	creating empty aisd_vision and aisd_motion
 LICENSE	creating empty aisd_vision and aisd_motion
 package.xml	creating empty asid_hearing and aisd_speaking packages
 setup.cfg	creating empty aisd_vision and aisd_motion
 setup.py	completing the package.xml and setup.py TODOs

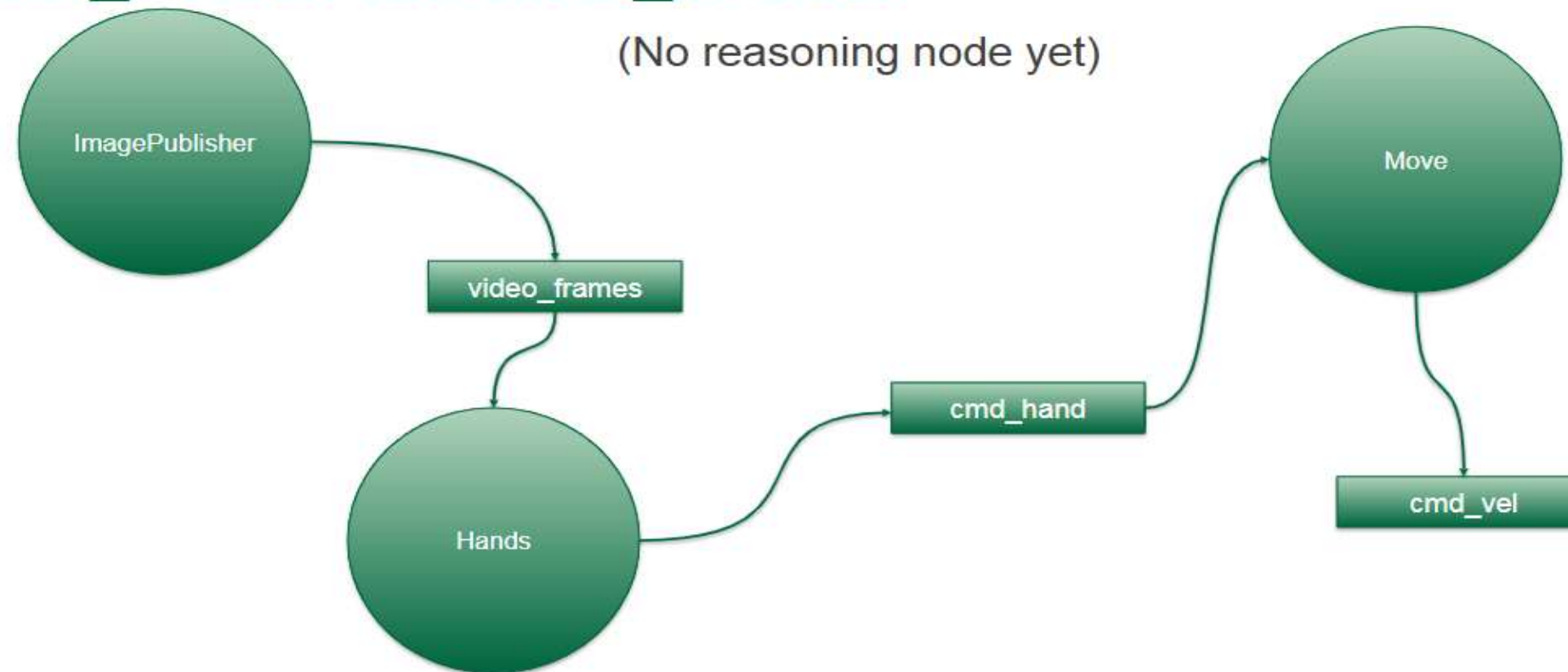
Notes in Assignment 2

- You need to build Three nodes

Name	Last commit message
..	
__init__.py	creating empty aisd_vision and aisd_motion
hands.py	adding comments to the codes, hand.py and move.py
image_publisher.py	move.py ,motion/package.xml hands.py and image_publishers modified

aisd_vision and aisd_motion

(No reasoning node yet)



Notes in Assignment 2

- Recall how to add dependencies to your ROS2 Packages and edit the setup.py

The `rosdep` command references `package.xml`, and that command will later be used to actually add the packages to the workspace overlay. For now, edit the `aisd_vision/package.xml` file, to add the dependencies. While the file is open, you can also set the Description, Maintainer, and License elements (no license):

```
<depend>roscpp</depend>
<depend>image_transport</depend>
<depend>cv_bridge</depend>
<depend>sensor_msgs</depend>
<depend>std_msgs</depend>
<depend>aisd_msgs</depend>
<depend>python3-mediapipe-pip</depend>
```

Now we will set up the entry points for the `aisd_vision` package. This is handled in the `aisd_vision/setup.py` file. Edit this file, find the `entry_points` variable, and change it to

```
entry_points={
    'console_scripts': [
        'image_publisher = aisd_vision.image_publisher:main',
        'hands = aisd_vision.hands:main',
    ],
},
```


Notes in Assignment 2

-Recall the sample node that we did last time. You need to change this part.

```
my_py_pkg > my_first_node.py >...
1  #!/usr/bin/env python3
2
3  import rclpy # import of the python interface with ros2
4  from rclpy.node import Node # use the class called Node in rclpy
5
6
7  class MyNode(Node): # define the MyNode class
8      def __init__(self): # initialize the class
9          super().__init__("py_test") # call the parent class constructor from the Node class
10         self.counter_ = 0 # initialize the counter value to be one
11         self.get_logger().info("Py Node started") # initialize the log
12         self.create_timer(0.5, self.timer_callback) # initialize a timer with 2 hertz to do timer_ca
13
14         def timer_callback(self): # define the timer_callback function
15             self.counter_ += 1 # add +1 to the counter
16             self.get_logger().info("Hello" + str(self.counter_)) # write Hello + new counter value
17
18
19
20
21 def main(args=None): # define the main function
22     rclpy.init(args=args) # initialize the ros2 commuincation
23     node = MyNode() # create an instance of the class called MyNode (we need to build it)
24     rclpy.spin(node) # keep the node running untill we kill it
25     rclpy.shutdown() # shutdown the ros2 commuincation
26
27 if __name__ == "__main__": # standard python line to execute the main function
28     main()
```

Notes in Assignment 2

- Follow the Doc of the assignment 2 to see how you can write the nodes. For exmpale the hand node

```
INDEX_FINGER_TIP = 8
PINKY_FINGER_TIP = 20
results = myhands.process(image)
if results.multi_hand_landmarks:
    #publish the hand position in terms of index finger and pinky
    msg = Hand()
    msg.xpinky = results.multi_hand_landmarks[0].landmark[PINKY_FINGER_TIP].x
    msg.xindex = results.multi_hand_landmarks[0].landmark[INDEX_FINGER_TIP].x
    if self.hand_publisher.get_subscription_count() > 0:
        self.hand_publisher.publish(msg)
    else:
        self.get_logger().info('waiting for subscriber')
```

Notes in Assignment 2

- The final Hand node should be as follows:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2
import mediapipe as mp
mp_hands = mp.solutions.hands
from aisd_msgs.msg import Hand

class HandPS(Node):
    def __init__(self):
        super().__init__('hand_commander')
        self.subscription = self.create_subscription(
            Image,
            'video_frames',
            self.listener_callback,
            10) #subscribes to the video_frames topic which was created by image_publisher node

        self.br = CvBridge()
        self.hand_publisher = self.create_publisher(Hand, 'cmd_hands', 10) #creates the topic cmd_hands to send message to move node

        self.subscription # prevent unused variable warning

    def listener_callback(self, msg):

def main(args=None):
    rclpy.init(args=args)

    hands = HandPS()

    rclpy.spin(hands)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    hands.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
```

Notes in Assignment 2

- The final Hand node should be as follows:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2
import mediapipe as mp
mp_hands = mp.solutions.hands
from aisd_msgs.msg import Hand

class HandPS(Node):
    def __init__(self):
        super().__init__('hand_commander')
        self.subscription = self.create_subscription(
            Image,
            'video_frames',
            self.listener_callback,
            10) #subscribes to the video_frames topic which was created by image_publisher node

        self.br = CvBridge()
        self.hand_publisher = self.create_publisher(Hand, 'cmd_hands', 10) #creates the topic cmd_hands to send message to move node

        self.subscription # prevent unused variable warning

    def listener_callback(self, msg):

def main(args=None):
    rclpy.init(args=args)

    hands = HandPS()

    rclpy.spin(hands)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    hands.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
```

Notes in Assignment 2

- The final motion node should be as follows:

Here is the code for the callback. Add comments to this code in order to show that you understand what each line is doing:

```
def listener_callback(self, msg):  
  
    angle = 0.0  
    linear = 0.0  
    if msg.xindex > 0.55:  
        self.get_logger().info('right')  
        angle = -0.1  
    elif msg.xindex < 0.45:  
        self.get_logger().info('left')  
        angle = 0.1  
    else:  
        angle = 0.0  
    if msg.xindex > msg.xpinky:  
        self.get_logger().info('come')  
        linear = 0.5  
    else:  
        self.get_logger().info('stay')  
        linear = 0.0  
    twist = Twist()  
    twist.linear.x = linear  
    twist.angular.z = angle  
    if self.vel_publisher.get_subscription_count() > 0:  
        self.vel_publisher.publish(twist)  
    else:  
        self.get_logger().info('waiting for subscriber')
```

OpenAI Whisper

- Open-source automatic speech recognition (ASR) system released by OpenAI Trained on 680,000+ hours of multilingual and multitask supervised data. Check this link: <https://openai.com/index/whisper/>

```
# 1) Load a pre-trained Whisper model
# Options: "tiny", "base", "small", "medium", "large"
# Smaller = faster, larger = more accurate
model = whisper.load_model("small")

# 2) Path to your audio file (wav, mp3, m4a, etc.)
audio_path = "example.wav"

# 3) Transcribe the audio
result = model.transcribe(audio_path)

# 4) Print the recognized text
print("Transcription:")
print(result["text"])
```


OpenAI Whisper

Multitask training data (680k hours)

English transcription

- 🗣️ "Ask not what your country can do for ..."
- 📝 Ask not what your country can do for ...

Any-to-English speech translation

- 🗣️ "El rápido zorro marrón salta sobre ..."
- 📝 The quick brown fox jumps over ...

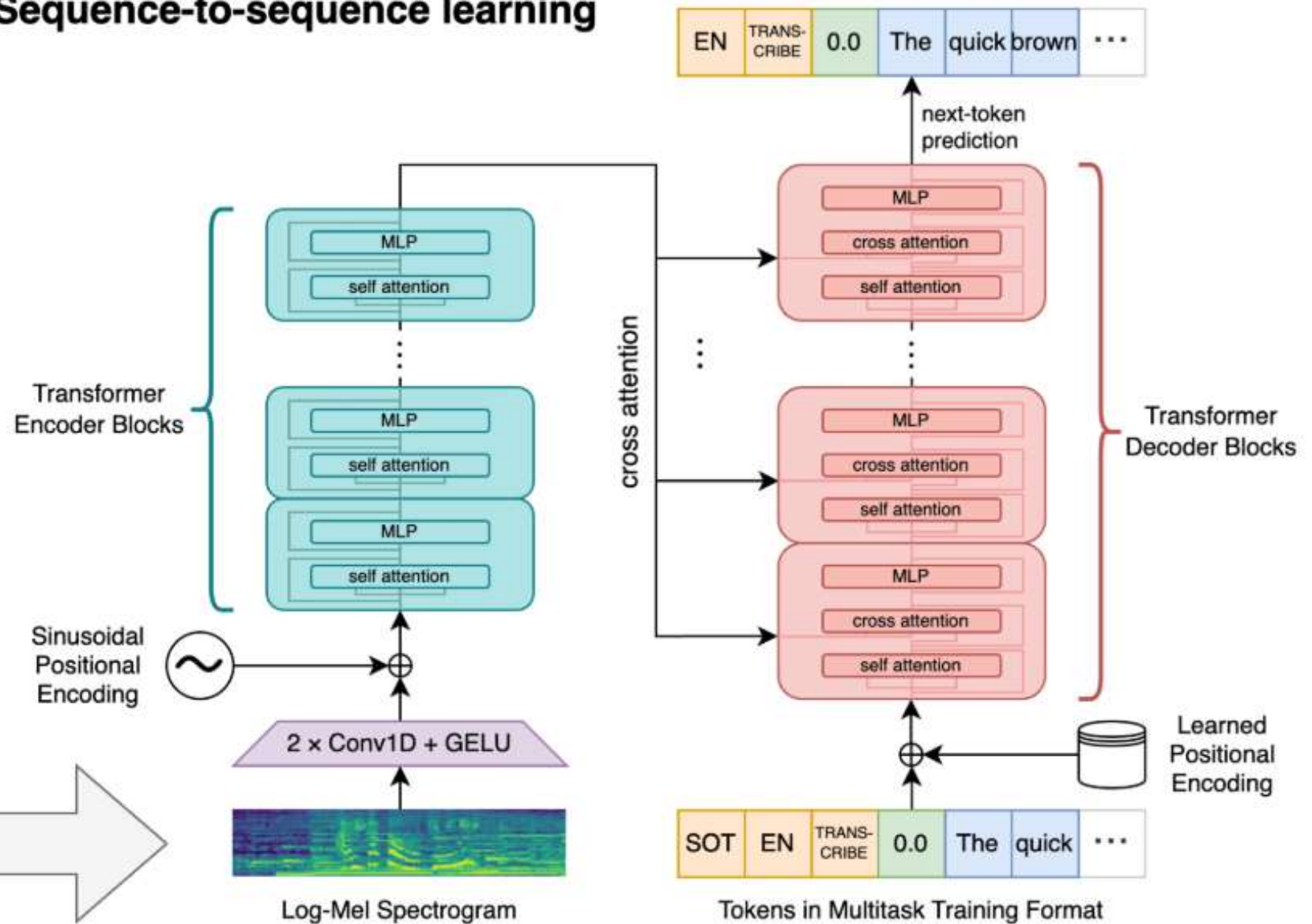
Non-English transcription

- 🗣️ "언덕 위에 올라 내려다보면 너무나 넓고 넓은 ..."
- 📝 언덕 위에 올라 내려다보면 너무나 넓고 넓은 ...

No speech

- 🔊 (background music playing)
- 📝 ∅

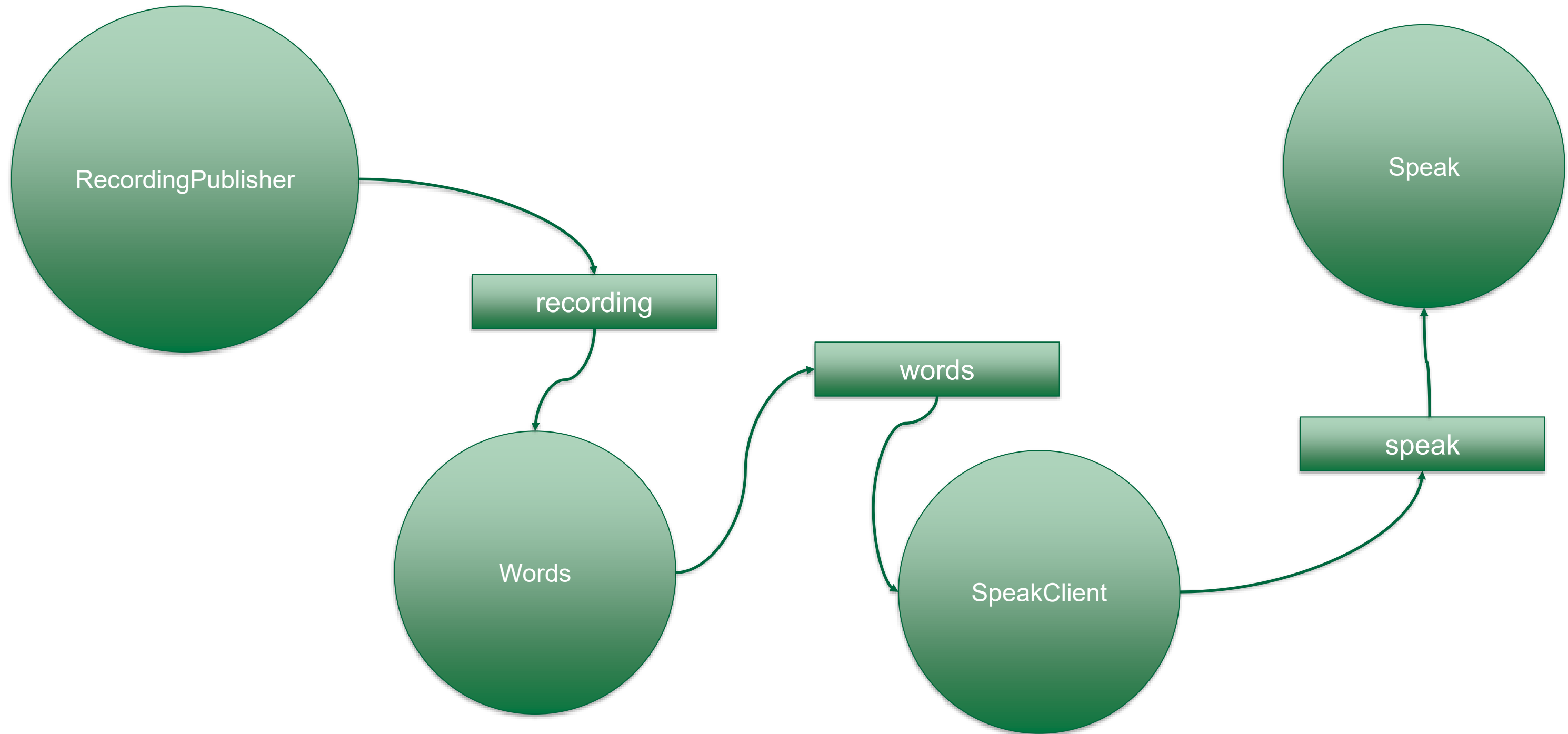
Sequence-to-sequence learning



ROS2 Workspace additions (Assignment 3)

- aisd_hearing
 - RecordingPublisher node
 - Publisher String messages on recording topic
 - WordsPublisher node
 - Subscriber to String messages on recording topic
 - Publisher of String messages on words topic
 - SpeakClient node
 - Subscriber to String messages on words topic
 - Client of SpeakService Speak.srv on speak topic
- aisd_speaking
 - SpeakService node
 - Service for Speak.srv (String words/String response) on speak topic

aisd_hearing and aisd_speaking



aisd_hearing package.xml dependencies:

Installed from command line:

```
sudo apt install alsa-utils
```

```
pip install git+https://github.com/openai/whisper.git
```

```
pip install pydub
```

In aisd_hearing/package.xml:

```
<depend>rclpy</depend>
```

```
<depend>std_msgs</depend>
```

```
<depend>python3-pyaudio</depend>
```

```
<depend>portaudio19-dev</depend>
```

```
<depend>python3-pip</depend>
```

aisd_hearing setup.py entry_points

```
entry_points={  
    'console_scripts': [  
        'recording_publisher = aisd_hearing.recording_publisher:main',  
        'words_publisher = aisd_hearing.words_publisher:main',  
        'speak_client = aisd_hearing.speak_client:main',  
    ],  
},
```

aisd_speaking package.xml dependencies:

Installed from command line:

```
pip install gtts
```

In aisd_hearing/package.xml:

```
<depend>rclpy</depend>
```

```
<depend>std_msgs</depend>
```

```
<depend>aisd_msgs</depend>
```

```
<depend>ffmpeg</depend>
```

aisd_speaking setup.py entry_points

```
entry_points={  
    'console_scripts': [  
        'speak = aisd_speaking.speak:main',  
    ],  
},
```